



UDB Protocol Plus

A modernization foundation for products that need to turn legacy systems, domain protocols, and fragmented backends into fast, governed, configurable, SDK-ready applications.

• UDB + HL7V2

• UDB + FHIR

• UDB + MQTT

• UDB + OPC UA

• PROTO-DRIVEN CONTRACTS

MODERNIZE

Wrap legacy apps with governed APIs, SDKs, events, and integrations.

EXTEND

Add domain protocol packs without rebuilding the application foundation.

SECURE

Built-in enterprise security, identity, authorization, audit, and evidence.

SCALE

Serve healthcare, industrial, IoT, government, and enterprise software from one broker core.

PROTOCOL PACK EXECUTION MODEL

Project proto

Application entities, events, services, security annotations, and SDK surfaces.

Protocol pack

HL7v2, FHIR, MQTT, OPC UA, and future packs add parsers, dictionaries, profiles, transports, and validators.

UDB runtime

Descriptor loading, authz, routing, canonical storage, outbox, CDC, migrations, and service dispatch.

SDKs & APIs

Go, Python, TypeScript, Java, C#, PHP, gRPC, generated docs, and compatibility manifests.

Operations

Health, audit, replay, DLQ, tenant/project isolation, pack versioning, and evidence reports.

UDB turns legacy software into a modern governed platform.

Most organizations are not starting from a clean greenfield system. They have old databases, isolated services, hospital interfaces, device streams, file drops, scripts, and operational workflows that still run the business. UDB gives those assets a modern contract layer without forcing every project to rebuild security, APIs, SDKs, storage, events, and integration logic.

01

01

Modernize without a rewrite

Expose legacy data and services through typed proto contracts, generated SDKs, and governed broker APIs while existing systems continue to operate.

02

Move faster with configuration

Projects define entities, services, backends, auth rules, events, and protocol pack choices as platform contracts instead of repeated application glue.

03

Enterprise security by default

Tenant, project, identity, scope, purpose, policy, audit, provenance, and evidence are platform concerns, not optional code paths.

04

Extend across domains

The same UDB core can support healthcare, industrial automation, IoT telemetry, government systems, and enterprise applications through selectable packs.

BEFORE UDB

Every product becomes its own integration platform.

Teams build one-off API layers, custom auth filters, fragile ETL jobs, message parsers, transport handlers, SDK wrappers, audit tables, and operational scripts. The result is slow delivery, inconsistent security, and expensive maintenance.

AFTER UDB

The broker becomes the stable foundation.

Products model the contract once, choose the needed backends and protocol packs, and let UDB provide the API surface, runtime enforcement, generated clients, eventing, migrations, and operational evidence.

Sales message: UDB is not only a database broker. It is the foundation layer that lets a legacy or fragmented product become configurable, extensible, secure, and integration-ready.

One foundation for regulated, operational, and protocol-heavy products.

UDB is valuable when customers need a product to absorb different data sources, protocols, and deployment models without losing control. Protocol Plus turns domain integration into a packaged capability instead of a consulting project.

CUSTOMER ENVIRONMENT	COMMON PAIN	WHAT UDB CHANGES	BUYER OUTCOME
Healthcare	HL7v2 feeds, FHIR APIs, patient data, lab results, imaging workflows, audits, and interface variation.	Protocol packs parse, validate, map, store, acknowledge, and expose clinical traffic through governed UDB contracts.	Legacy and modern healthcare integration becomes safer, faster, and easier to sell.
Industrial and SCADA	OPC UA nodes, MQTT streams, plant devices, alarms, commands, certificates, and time-series telemetry.	UDB brings device identity, protocol metadata, command authorization, evidence, storage, and SDKs into one platform.	Operational data becomes usable by dashboards, analytics, apps, and automation without bypassing governance.
Government and public sector	Multiple agencies, legacy systems, strict access control, reporting, long retention, auditability, and controlled change.	Tenant/project isolation, policy, provenance, migration gates, evidence reports, and generated APIs reduce hidden integration risk.	Modern services can be built over existing assets with clear accountability and upgrade paths.
Enterprise SaaS and internal platforms	Many databases, scattered services, customer-specific integrations, SDK drift, and repeated backend plumbing.	Proto-first contracts, multi-backend routing, native services, events, SDK generation, and optional packs standardize delivery.	Teams ship configurable product capabilities instead of rebuilding infrastructure per customer.

For executives

UDB protects existing investment while creating a path to modern, packaged, protocol-aware products.

For engineering

UDB removes repeated infrastructure work and gives teams a typed contract, runtime, SDK, and operational model.

For operations

UDB makes integration traffic observable, auditable, replayable, governed, and easier to support across customers.

UDB turns protocol integration into a governed product layer.

Most projects need a data platform and one or more domain protocols. UDB Core supplies the broker, storage, authorization, native services, SDKs, migrations, and events. UDB Plus adds protocol packs so the same project can opt into HL7v2, FHIR, MQTT, OPC UA, or future protocols through a consistent runtime shape.

03

77

DataBroker RPCs

One public data plane for records, objects, vectors, cache, CDC, transactions, catalog, migration, and health operations.

15

Native services

Internal control plane for authn, authz, IdP, tenants, notifications, analytics, storage, assets, WebRTC, and policy.

18

Backend kinds

SQL, cache, vector, object, document, graph, search, and columnar stores behind one broker contract.

6

SDK families

Go, Python, TypeScript/Node, Java, C#, and PHP/Laravel generated from descriptor-aware contracts.

PROBLEM UDB SOLVES

Integration logic is usually duplicated in every product.

Each project repeats parser rules, transport retries, schema mapping, tenant filtering, security, audit, storage, SDKs, and operational health. As protocols multiply, the application becomes the integration engine.

UDB ANSWER

Move the contract into proto and the protocol into packs.

Application teams model their domain once. Protocol packs contribute dictionaries, codecs, validation, transport semantics, mappings, evidence, and SDK-aware service surfaces without weakening the core broker boundary.

The product idea: UDB Core is the required broker foundation. UDB Plus enables protocol packs per project: UDB + HL7v2, UDB + FHIR, UDB + MQTT, UDB + OPC UA, and additional packs later.

Core platform plus protocol packs.

The tiering is intentionally simple: every customer gets the broker foundation; Plus customers attach protocol packs to the projects that need domain-specific interoperability.

04

OFFERING	WHAT IT INCLUDES	BEST FIT	OPERATIONAL RESULT
UDB Core	Proto annotations, DataBroker API, native services, backend routing, tenant/project security, SDK generation, migrations, CDC, audit, health, and CLI.	Applications that need one typed API across many stores and services.	Less backend glue, one contract, and one runtime policy point.
UDB Plus	Protocol-pack runtime, pack manifests, dictionaries, profile validation, transport adapters, canonical artifacts, evidence, generated docs, and SDK protocol indexes.	Products that need domain protocols in addition to normal app data.	The project opts into protocol support without becoming a custom integration engine.
UDB + HL7v2	HL7v2 parser, serializer, ACK facade, dictionary snapshots, version selection, table/datatype/segment/message validation, MLLP-ready transport boundary, and seed-pack tooling.	Healthcare interfaces, labs, EHR integration, ADT, ORM, ORU, SIU, MDM, DFT, VXU, and ACK workflows.	Legacy HL7v2 flows become governed, auditable, and SDK-accessible.
UDB + FHIR	FHIR resource modeling, REST/bundle handling, profile-aware validation, terminology binding, search/query mapping, and evidence linkage.	Modern healthcare APIs, patient access, care coordination, analytics, payer/provider exchange, and app ecosystems.	FHIR resources connect to UDB storage, policy, SDKs, and event streams.
UDB + MQTT	Topic modeling, device identity, payload schemas, QoS posture, retained-state handling, telemetry ingestion, command routing, and time-series/event projection.	IoT devices, sensors, meters, gateways, telemetry systems, and edge-to-cloud workloads.	Device streams land in governed project data with tenant, device, and topic context.
UDB + OPC UA	Namespace/node modeling, browse/import, datatype mapping, monitored items, method calls, alarms/events, certificates, and industrial evidence trails.	Industrial automation, SCADA, building systems, process control, and machine telemetry.	Operational technology data becomes broker-governed without losing source node identity.

Everything starts from proto.

UDB uses protobuf descriptors as the source of truth for the application contract. Protocol packs extend that model with facts, intermediate representations, validation plans, and runtime services.

05

1. Model

Project protos define entities, events, services, security, storage annotations, and SDK surfaces.

2. Compile

UDB builds a descriptor manifest, catalog, SQL, SDK manifests, protocol indexes, and generated docs.

3. Attach packs

Protocol packs contribute dictionaries, profiles, codecs, transport rules, evidence, and service facades.

4. Execute

The broker enforces tenant/project context, authorization, routing, admission, backend execution, audit, and events.

5. Operate

Health, drift, replay, DLQ, migration, pack compatibility, and SDK conformance are checked through the same control plane.

Descriptor-driven

- No handwritten API inventory.
- Generated SDKs and docs derive from proto.
- Method, field, table, event, and protocol metadata share one contract.

Tenant/project scoped

- Every call carries tenant, project, purpose, user, scopes, service identity, and correlation id.
- Runtime rejects unresolved scope instead of guessing.
- Pack operations inherit the same security boundary.

Backend-neutral

- Postgres, MySQL, SQLite, SQL Server, ClickHouse, Redis, MongoDB, MinIO/S3, Neo4j, vector stores, and more.
- Protocol data can land in project-owned canonical models.

A pack is more than a parser.

Each protocol pack carries the protocol facts, executable runtime behavior, service boundaries, generated client affordances, and evidence needed to operate safely.

LAYER	DICTIONARY	RUNTIME	SERVICES	EVIDENCE
Facts	Versions, entities, events, messages, segments, resources, datatypes, fields, tables, profiles, nodes, topics, bindings.	Compiled snapshots, lookup tables, validation plans, codec hints, compatibility manifests.	Request/response protos, service facades, generated SDK indexes, project opt-in metadata.	Source provenance, hashes, coverage summaries, diffs, curation reports, audit posture.
Behavior	Version-specific variance and profile-specific constraints.	Parse, serialize, normalize, validate, map, route, acknowledge, subscribe, publish, browse, and command.	Gateway operations, batch jobs, transport endpoints, subscriptions, replay, and health checks.	Issue reports, validation paths, source references, deterministic fingerprints, replayable errors.
User value	Customers do not hand-code protocol dictionaries.	Projects get protocol behavior without embedding a separate engine.	SDK users call a coherent UDB protocol surface.	Operations can explain exactly what happened and why.

Dictionary

Protocol versions, messages, resources, tables, datatypes, topics, nodes, and profile metadata.

Core

Canonical intermediate representation, references, paths, cardinality, vocab bindings, and provenance.

Transport

MLLP, HTTP/FHIR, MQTT broker, OPC UA secure channel, retries, framing, acknowledgments, and sessions.

Validation

Structural checks, datatype checks, vocabulary checks, profile rules, correction reports, and issue severity.

Legacy healthcare messaging, handled as a first-class protocol.

HL7v2 is not a single schema. It is a family of versions, trigger events, message structures, segments, datatypes, escape rules, table bindings, acknowledgments, and transport conventions. UDB + HL7v2 treats those differences as data and runtime contracts.

2.x

Version-aware dictionaries

Compiled snapshots select behavior from MSH-12, including message, segment, datatype, table, and structure metadata.

ER7

Lossless parser

Preserves raw bytes, MSH separator semantics, segment terminators, repetitions, components, subcomponents, null fields, and non-UTF8 diagnostics.

ACK

Messaging facade

Parse, resolve, validate, serialize, and generate application ACKs behind a protocol-ready service boundary.

HL7V2 CAPABILITY	WHAT UDB PROVIDES	CLIENT VALUE
ADT, ORM, ORU, SIU, MDM, DFT, VXU, ACK	Message-family resolution from MSH-9 and MSH-12, with compiled dictionary lookup.	One platform can ingest administrative, order, result, scheduling, document, billing, immunization, and acknowledgment flows.
Segments and datatypes	Required segment checks, field cardinality, datatype references, component metadata, table references, and warning/error severity.	Validation becomes explainable instead of buried inside interface scripts.
Transport boundary	MLLP-ready framing and commit ACK posture can be separated from HL7 application ACK semantics.	Deployments can integrate with interface engines without mixing transport and message semantics.
Seed pack	UDB-owned UPF source, compiled snapshots, SQLite support, reports, deterministic fingerprints, and curation workflow.	Protocol behavior can be rebuilt, audited, and upgraded without private third-party research folders.

FHIR resources connected to storage, policy, and events.

UDB + FHIR uses the same pack architecture for modern healthcare APIs: resources, profiles, extensions, terminology bindings, bundles, search behavior, and provenance artifacts can connect to project-owned data models and SDKs.

08

Resources

Patient, Encounter, Observation, DiagnosticReport, Medication, Procedure, ImagingStudy, Organization, Practitioner, and custom profiles.

Profiles

ImplementationGuide packaging, StructureDefinition constraints, extensions, slices, cardinality, and profile-specific validation.

Terminology

ValueSet and CodeSystem binding, binding strength, code validation, and versioned terminology provenance.

Operations

Bundle ingest, REST facade, search mapping, resource history, subscriptions, provenance, and consent-aware authorization.

FHIR request

REST, bundle, or SDK call enters the protocol facade.

Profile resolve

Resource type, version, profile, and extensions determine validation posture.

Policy check

Tenant, patient, project, purpose, user, and scopes are checked centrally.

Store/project

Resources map to canonical UDB records, object payloads, events, or project services.

Evidence

Validation, provenance, changes, and event history stay queryable.

Device telemetry and command streams with project governance.

UDB + MQTT is for projects where devices, sensors, gateways, and edge services publish data faster than normal CRUD APIs can model cleanly. The pack turns topics and payloads into typed, policy-aware project data.

MQTT AREA	PROTOCOL PACK RESPONSIBILITY	UDB PLATFORM RESPONSIBILITY
Topics	Topic patterns, wildcards, device groups, tenant/project topic namespaces, and route classification.	Catalog mapping, authorization, event contracts, SDK protocol index, and generated documentation.
Payloads	JSON, protobuf, binary, or custom codecs; schema validation; unit metadata; timestamp and quality handling.	Canonical storage, time-series projection, outbox publication, object archival, and replay.
Sessions	Client identity, retained messages, will messages, QoS posture, duplicate handling, and reconnect behavior.	Tenant/device identity, policy decisions, audit, health, and operational metrics.
Commands	Downlink topics, command correlation, acknowledgment mapping, and delivery reports.	RBAC/ABAC/ReBAC checks, idempotency, audit chain, saga coordination, and DLQ.

IoT products

- Device telemetry ingestion.
- State snapshots and event history.
- Command routing with policy checks.

Facilities and meters

- Sensor values, counters, alarms, energy usage, and environment readings.
- Project-ready APIs for dashboards and reports.

Edge gateways

- Bridge industrial protocols into MQTT topics.
- Forward typed data to UDB while keeping source identity.

Industrial node models without losing broker governance.

UDB + OPC UA is designed for industrial and facility systems where data is organized by namespaces, nodes, datatypes, monitored items, methods, alarms, and secure channels.

Browse and import

Namespace discovery, node id references, object/variable/method classification, datatype mapping, and project model suggestions.

Monitor

Subscriptions, monitored items, sampling, queue policy, timestamps, status codes, alarms, and event payloads.

Control

Write requests, method calls, authorization gates, command receipts, replay posture, and audit evidence.

Secure

Certificates, endpoint policy, session identity, tenant/project mapping, and operational health reporting.

The OPC UA pack is the industrial counterpart to FHIR and HL7v2: preserve source identity, compile protocol metadata, expose safe service operations, and project the result into UDB-managed data, events, and SDKs.

INDUSTRIAL SCENARIO	HOW UDB HELPS	WHY IT MATTERS
SCADA dashboards	Expose live node values and alarms through authorized project APIs.	Operations teams get real-time visibility without direct plant-network coupling.
Predictive maintenance	Project machine telemetry into time-series, vector, document, and analytics backends.	Data science and reliability programs can use the same governed broker path.
Command workflows	Gate writes and method calls behind identity, scope, purpose, and audit controls.	Industrial control remains accountable and reviewable.

One facade for parse, validate, transform, transport, and store.

Protocol packs should feel native to SDK users. The service layer presents high-level operations while the runtime retains lossless artifacts, validation details, acknowledgments, mappings, and transport evidence.

11

Parser and serializer

- Lossless raw-byte capture.
- Canonical serialization.
- Version and delimiter awareness.
- Structured path access.

Validation and profile

- Structural validation.
- Datatype and vocabulary binding.
- Profile rules and issue severity.
- Corrections as reports, not mutation of parsed facts.

Transport and messaging

- MLLP and application ACK separation.
- FHIR HTTP/bundle behavior.
- MQTT session/topic/QoS mapping.
- OPC UA secure channel and subscriptions.

Mapping

Protocol paths map into project entities, events, services, and evidence objects without deleting source provenance.

Storage

Records, object payloads, search indexes, graphs, vectors, analytics, and time-series projections can share UDB routing.

SDKs

Generated clients expose project data plus protocol operations in Go, Python, TypeScript, Java, C#, and PHP/Laravel.

Protocol traffic receives the same controls as application data.

Healthcare and industrial protocols often arrive through side channels. UDB Plus keeps those channels inside the same tenant/project, authorization, audit, and evidence model as normal application APIs.

CONTROL AREA	UDB SUPPORT	PROTOCOL-PACK BENEFIT
Identity and tenant scope	Verified request context, tenant id, project id, user id, purpose, scopes, service identity, and correlation id.	HL7 feeds, FHIR calls, MQTT clients, and OPC UA sessions can be bound to real project boundaries.
Authorization	RBAC, ABAC, ReBAC, policy bundles, native access, and fail-closed checks.	Protocol writes, commands, subscriptions, and transformations can be authorized before execution.
Audit and evidence	Admin audit, event envelopes, provenance, hash chain support, validation reports, and deterministic fingerprints.	Every parse, validation, ACK, mapping, and command can explain what source data produced what result.
Reliability	Outbox, CDC, DLQ, replay controls, saga coordination, idempotency, and health checks.	Protocol messages can be retried, replayed, quarantined, or acknowledged with explicit semantics.
Deployment	Broker serving, CLI diagnostics, SDK generation, compatibility matrix, local dev, and native-service control plane.	Protocol capability becomes deployable infrastructure, not per-project scripts.

What customers buy.

UDB Protocol Plus is not just a technical runtime. It is a product packaging model that lets teams sell protocol readiness as a controlled project capability.

13

FOR HEALTHCARE SOFTWARE**One platform for HL7v2 and FHIR coexistence.**

- Connect legacy feeds and modern APIs without two unrelated integration stacks.
- Preserve message-level evidence while projecting into project-owned models.
- Offer SDKs and data APIs to product teams instead of raw interface-engine scripts.

FOR INDUSTRIAL AND IOT SOFTWARE**One platform for MQTT and OPC UA data paths.**

- Model devices, topics, nodes, commands, alarms, and telemetry through one broker.
- Route real-time data into storage, analytics, dashboards, alerts, and APIs.
- Apply identity, authorization, and audit to command and control operations.

Faster onboarding

Projects opt into a protocol pack instead of writing a full protocol integration subsystem.

Lower risk

Versioned dictionaries, validation reports, provenance, and deterministic builds reduce hidden integration behavior.

Better SDK story

Developers use the same SDK posture for normal data and protocol-specific workflows.

Enterprise fit

Security, tenant isolation, audit, health, migrations, and eventing are broker-owned rather than project-owned.

Positioning statement.

UDB is the core broker. UDB Plus is the protocol extension layer. Together they let a product declare: this project uses UDB + HL7v2, UDB + FHIR, UDB + MQTT, UDB + OPC UA, or any combination needed by the customer.

TECHNICAL BUYER

Protocol behavior becomes inspectable infrastructure.

Parser rules, dictionaries, profiles, transports, storage, and SDKs can be reviewed, tested, and upgraded through the same platform lifecycle.

PRODUCT BUYER

Protocol support becomes a sellable project capability.

Teams can package domain readiness without fragmenting the core platform or building one-off integration engines.

OPERATIONS BUYER

Protocol traffic becomes governable.

Tenant/project isolation, health, audit, DLQ, replay, provenance, and evidence reports apply to protocol flows.

Recommended go-to-market language: "UDB Core powers the broker. UDB Plus adds protocol packs. Choose the packs your project needs: healthcare messaging, FHIR APIs, IoT telemetry, industrial node models, and future protocol domains."

PACKAGE LABEL	CUSTOMER-FACING DESCRIPTION
UDB Core	Schema-driven multi-backend data broker with native control plane, SDKs, migrations, events, storage, identity, authorization, and audit.
UDB Plus	Protocol-pack runtime for regulated and operational integrations that need parse, validate, transform, transport, persist, acknowledge, and audit features.
UDB + HL7v2 / FHIR / MQTT / OPC UA	Project-selectable protocol capabilities delivered on top of the same UDB broker, SDK, security, and operations model.